

P2262R0

2020 Fall Library Evolution Poll Outcomes

Published Proposal, 2020-12-02

Author:

[Bryce Adelstein LeLach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1	Introduction
2	Summarized Poll Results
3	Summarized Outcomes
4	Detailed Poll Results and Selected Comments
4.1	Executor Polls
4.1.1	Poll 0
4.1.2	Poll 1
4.1.3	Poll 2
4.1.4	Poll 3
4.1.5	Poll 4
4.1.6	Poll 5
4.2	Other Polls

- 4.2.1 Poll 6
- 4.2.2 Poll 7
- 4.2.3 Poll 8

References

Informative References

§ 1. Introduction

In Fall 2020, the C++ Library Evolution group took a series of electronic decision polls [\[P2233R3\]](#). This paper provides the results of those polls and summarizes the results.

In total, 49 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Summarized Poll Results

Poll	SF	WF	N	WA	SA	Outcome
Poll 0: Remove implicit adaptation from [P0443R14] (Executors) by applying [P2235R0] to [P0443R14] , which will make <code>schedule</code> only take schedulers, make <code>execute</code> only take executors, make <code>sender</code> and <code>receiver</code> operations like <code>connect</code> only take senders and receivers, and add explicit adaptation from executor to scheduler (<code>make_scheduler_from_executor</code>) but not vice versa.	21	15	0	0	0	Unanimous consensus in favor.
Poll 1: Use one class for each individual trait instead of combined traits classes (<code>sender_traits</code> , etc) in [P0443R14] (Executors).	7	7	7	5	2	No consensus.
Poll 2: Remove <code>static_thread_pool</code> from [P0443R14] (Executors). It may be pursued in a follow-on proposal.	8	11	4	7	2	Weak consensus in favor.

Poll 3: Remove <code>any_executor</code> from [P0443R14] (Executors). It may be pursued in a follow-on proposal.	10	9	4	4	2	Consensus in favor.
Poll 4: Remove <code>any_executor::target</code> and <code>any_executor::target_type</code> from [P0443R14] (Executors).	12	10	4	2	0	Consensus in favor.
Poll 5: [P0443R14] (Executors) is sufficiently mature that we should aim to ship it in C++23.	10	12	7	10	1	Weak consensus in favor.
Poll 6: Send [P2212R1] (Relax Requirements for <code>time_point::clock</code>) to LWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	18	13	3	0	0	Strong consensus in favor.
Poll 7: Send [P2166R1] (Prohibit <code>basic_string</code> and <code>basic_string_view</code> Construction from <code>nullptr</code>) to LWG for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	15	18	2	2	1	Consensus in favor.
Poll 8: Send [P2161R2] (Remove Default Candidate Executor) to LWG for the Networking TS P-numbered Working Draft, classified as a focus work item ([P0592R4] bucket 1 item).	6	14	2	2	0	Consensus in favor.

§ 3. Summarized Outcomes

Revise [\[P0443R14\]](#) (Executors) as follows and return to Library Evolution for further review:

- Apply [\[P2235R0\]](#).
- Remove `static_thread_pool`. A separate follow-on proposal for `static_thread_pool` is encouraged.
- Remove `any_executor`. If `any_executor` is pursued in the future, `any_executor::target` and `any_executor::target_type` shall be removed.

We will continue on our planned course of aiming to ship [\[P0443R14\]](#) (Executors) in C++23, but a notable minority in Library Evolution are not convinced that [\[P0443R14\]](#) (Executors) is sufficiently mature. Library Evolution wants to see more field experience with [\[P0443R14\]](#) (Executors). P0443R14 (Executors) authors and advocates should take note of this. Focus on demonstrating field experience through implementations and usage, improving introductory material, minimizing the scope, resolving outstanding minor open issues, and developing wording to increase Library Evolution confidence in the maturity of [\[P0443R14\]](#) (Executors).

We encourage a separate follow-on proposal exploring individual traits versus combined traits classes in general, using [\[P0443R14\]](#) (Executors) as an example to gauge impact.

[\[P2212R1\]](#) (Relax Requirements for `time_point::clock`) is sent LWG for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

[\[P2166R1\]](#) (Prohibit `basic_string` and `basic_string_view` Construction from `nullptr`) is sent to LWG for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

[\[P2161R2\]](#) (Remove Default Candidate Executor) is sent to LWG for the Networking TS Working Draft, classified as a focus work item ([\[P0592R4\]](#) bucket 1 item).

§ 4. Detailed Poll Results and Selected Comments

§ 4.1. Executor Polls

The following C++ Library Evolution polls relate to executors ([\[P0443R14\]](#)), a proposed set of abstractions for asynchronously creating and managing execution agents.

§ 4.1.1. Poll 0

Remove implicit adaptation from [\[P0443R14\]](#) (Executors) by applying [\[P2235R0\]](#) to [\[P0443R14\]](#) which will:

- Make `schedule` only take schedulers.
- Make `execute` only take executors.
- Make sender and receiver operations like `connect` only take senders and receivers.
- Add explicit adaptation from executor to scheduler (`make_scheduler_from_executor`) but not vice versa.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
21	15	0	0	0

§ 4.1.1.1. Outcome

Unanimous consensus in favor. We will need to continue discussing explicit adaptation, the relationship between `scheduler` and `executor` and the implications of bifurcation on the ecosystem.

§ 4.1.1.2. Selected Comments

I feel unable to support moving this proposal forward since I still find it next to impossible to understand what's going on, despite extensive discussion and leading a review group.

— Did Not Vote

[Implicit adaptation] adaption was the biggest issue with [\[P0443R14\]](#)!

— Strongly Favor

This is a step in the right direction. I think additional simplifications are possible and desirable, but I am happy with this direction.

— Strongly Favor.

The circularity between `execution::execute` and `execution::connect` is complicated and must be eliminated. However, the resulting disconnection between `executors` and `schedulers` is a bug that must be addressed.

— Strongly Favor

This is fine, though it may result in some awkwardness when forming overloads that accept both a `scheduler` and an `executor`.

— Weakly Favor

I think that the [\[P2235R0\]](#) suggestions are good overall. Removing implicit adaptations should make things clearer. Currently there are places where the behavior of customization points is not obvious due to all of those implicit adaptation features. However, I still think that there are different opinions about which notion is the base one: `scheduler` or `executor`. To me it's `scheduler`. I believe we already discussed that fire-and-forget semantics can be implemented via schedulers. On the other hand, executors don't support chaining and error handling in a manner how schedulers do that. That's why schedulers look more fundamental IMO. I could easily imagine `make_executor_from_scheduler` method that returns executor, which ignores error channel or terminates. Overall: what I mean by my comment is the explicit adaptation functions set is still open question to me. And I am raising which one is more fundamental because I see the possible implications to other part of C++ library (e.g. C++17 parallel algorithms).

— Weakly Favor

I approve of the simplification. I disapprove of the resulting bifurcation. There will be two async worlds - libraries that deal in executors and libraries that deal in schedulers. The status quo required library authors to consider and support both - now authors will focus on one and ignore the other. Given this change LEWG needs to decide which will survive and remove the other one.

— Weakly Favor

§ 4.1.2. Poll 1

Use one class for each individual trait instead of combined traits classes (`sender_traits`, etc) in [\[P0443R14\]](#) (Executors).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
7	7	7	5	2

§ 4.1.2.1. Outcome

No consensus. We encourage a separate follow-on proposal exploring individual traits versus combined traits classes in general, using [\[P0443R14\]](#) (Executors) as an example to gauge impact.

We've been moving towards single traits in general. They are easier to specialize, and are more future-proof.

— Strongly Favor

`sender_traits`' implementation is complicated. Separate traits are easier to implement and use in metaprogramming.

— Strongly Favor

Combined trait classes are often burdensome as extensive experience with `numeric_traits` or especially `iterator_traits` has shown.

— Strongly Favor

I feel this is a preferable approach as it makes the traits more extensible and aligns with the precedent set by more recently introduced traits.

— Weakly Favor

I think it's worth thinking about using one class per trait, but I don't think this should delay the proposal.

— Neutral

Can live with both

— Neutral

I'm conflicted. I like them grouped when they're being used for their original intent. It helps when learning about what needs to be defined. If trying to reuse a trait for something else, then it can be unfortunate. That case should probably define a specific trait for its own use.

— Weakly Against

For associated types, current practice is for one trait per concept (`iterator_traits`, `allocator_traits`, `incrementable_traits`, `readable_traits`). This also brings down total number of class template instantiations. There are exceptions (`is_invocable`, `invocable_result`, etc.), so I am only mildly opposed.

— Weakly Against

This provides the customiser with more ways to get an already complicated thing wrong

— Strongly Against

§ 4.1.3. Poll 2

Remove `static_thread_pool` from [\[P0443R14\]](#) (Executors). It may be pursued in a follow-on proposal.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
8	11	4	7	2

§ 4.1.3.1. Outcome

Weak consensus in favor. Remove `static_thread_pool` from [\[P0443R14\]](#) (Executors). A separate follow-on proposal is encouraged as a number of people feel it is important.

§ 4.1.3.2. Selected Comments

No need to tie `static_thread_pool` to executors, as the discussion is still ongoing and it isn't a fundamental necessity.

— Strongly Favor

[\[P0443R14\]](#) conflates too many things. The thread pool is not fundamental to the concept of execution, and may not be the right default thread pool API for us to provide users anyway (a parallel forward progress system thread pool might be a better default).

— Strongly Favor

[\[P0443R14\]](#) does not define a concept for execution contexts like `static_thread_pool`. We are not ready to define the shape of an execution context - it is purely implementation defined for now.

— Strongly Favor

`static_thread_pool` is niche but doesn't look that way, especially if it's the only execution resource provided in C++23. We would be setting the community up to fail in the future when we standardize `system_executor`.

— Weakly Favor

There needs to be some concrete execution context available at the time that executors/schedulers become available in the standard. However because of the length/complexity of P0443, important sections, such as `static_thread_pool`, can get lost in discussions. For this reason, I think it would be better to isolate it in its own proposal so that it gets the full attention it requires.

— Weakly Favor

It's still unclear to me how `static_thread_pool` is problematic, but I'm not against it if it builds consensus.

— Neutral

I don't care for designs that are all framework, but no concrete implementation. `static_thread_pool` seems useful in portable code, and it doesn't seem terribly onerous to specify or implement.

— Weakly Against

I feel it is important to have an initial thread pool executor available with executors when they are first introduced to C++ as it is a feature high in demand and provides the first standardized execution context, setting the convention for the execution context - executor relationship, so I would prefer the `static_thread_pool` to remain in [\[P0443R14\]](#).

— Weakly Against

The `static_thread_pool` functionality is simple, succinct, mature, and satisfies a very common use case.

— Strongly Against

§ 4.1.4. Poll 3

Remove `any_executor` from [\[P0443R14\]](#) (Executors). It may be pursued in a follow-on proposal.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
10	9	4	4	2

§ 4.1.4.1. Outcome

Consensus in favor. Remove `any_executor` from [\[P0443R14\]](#) (Executors). If advocates for `any_executor` still wish to pursue it, they should bring a separate proposal which contains detailed motivation and discussion of the ABI impacts and risks of `any_executor`.

§ 4.1.4.2. Selected Comments

There are too many open questions about whether wrapping just any executor or more broadly using reference wrappers. It also added a lot of questions where we seem to be defining mechanisms purely to support this type, yet it is not fundamental to the execution concepts [\[P0443R14\]](#) should be focused on. It is important we have such a thing, but it should build on what is in [\[P0443R14\]](#) rather than driving it.

— Strongly Favor

Separating `any_executor` from [\[P0443R14\]](#) will allow it to make progress faster.

— Weakly Favor

`any_executor` is complicated enough that keeping it in [\[P0443R14\]](#) would slow things down.

— Weakly Favor

`any_executor` has long complicated many aspects of the design of executors, properties in particular. While I understand and sympathize with the use case, I don't view this as essential to executors and would be willing to ship a standard without `any_executor`. We don't need a type erasure mechanism for everything in the standard. There are many facilities in the standard library, such as iterators and ranges, which could have a type erasure mechanism, but do not. Additionally, standardizing type erasure facilities almost always leads to ABI issues down the road. Once we ship an `any_executor`, it will be much harder to modify and fix problems in both executors and properties down the line.

— Strongly Favor

Shipping it with executors will lock us into an ABI and I am very VERY much against that.

— Strongly Favor

My main concern is ABI. If we get a proposal that adds a version field or something similar to it so that we can improve `any_executor` in the future, then I would be fine leaving it in the main paper.

— Weakly Favor

I think if it's possible to make the things separated they probably should be split. It allows them to move in parallel without stopping each other. Furthermore, the Executors unified proposal looks too big. If we can make it simpler to read and to understand, let's do that.

— Weakly Favor

There is enough disagreement around `any_executor` that it should be a separate proposal.

— Weakly Favor

The erasure for the concepts are hard to specify and review, and I believe this should be pursued with paper that presents concrete use case (Networking TS).

— Strongly Favor

I’ve always been neutral on whether or not executors should contain a type erasure mechanism from the start, but others were so strongly opinionated in the past that we added it. I’m in favor of whatever builds the most consensus.

— Neutral

I don’t know what type I’m supposed to use for non-generic code if this facility is removed.

— Weakly Against

I believe that the polymorphic wrapper is a major use case for a number of applications and it should go together with the base specification.

— Weakly Against

`any_executor` is a convenient base for implementing polymorphic executors where two domains must intercommunicate. Furthermore, this would impact negatively on the Networking TS as IO objects would be required to know their executor types. This negatively affects: 1. code reuse with coroutines, and 2. compile times.

— Strongly Against

A polymorphic executor is on the critical path for networking + [\[P1322R2\]](#). Deferring the choice of target executor until runtime is an extremely common requirement.

— Strongly Against

§ 4.1.5. Poll 4

Remove `any_executor::target` and `any_executor::target_type` from [\[P0443R14\]](#) (Executors).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
12	10	4	2	0

§ 4.1.5.1. Outcome

Consensus in favor. We have decided to remove `any_executor` will be removed from [\[P0443R14\]](#), but if it is pursued in another proposal, `any_executor::target` and `any_executor::target_type` shall be removed.

§ 4.1.5.2. Selected Comments

These methods come with an unavoidable overhead which we should not impose on users. If you need this functionality, it has been demonstrated that you can achieve it using properties. Adding `target` and `target_type` to `std::function` was a mistake that we should not repeat. Removing these methods is consistent with other similar facilities that are in flight (`any_invocable` and `function_ref`).

— Strongly Favor

Including them breaks "don't pay for what you don't use" by forcing RTTI to be generated and stored in `any_executor` even if never called, and it's possible to replicate the semantics removed here with properties.

— Strongly Favor

The functionality that relies on this in the Networking TS can be re-implemented in another way. The current interface defeats the purpose of polymorphism by asking "what are you?", rather than the pertinent question of "can you fulfill my requirements?"

— Strongly Favor

I believe that `any_executor::target` has utility and have used it in production code. `any_executor::target_type` less so and seems to be the point of contention due to RAII overhead. Given that these removals were coupled in the poll question I have to vote against although I would be weakly in favor of removing `any_executor::target_type`.

— Weakly Against

§ 4.1.6. Poll 5

[\[P0443R14\]](#) (Executors) is sufficiently mature that we should aim to ship it in C++23.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
10	12	7	10	1

§ 4.1.6.1. Outcome

Weak consensus in favor. We will continue on our planned course of aiming to ship [\[P0443R14\]](#) (Executors) in C++23, but a notable minority in Library Evolution are not convinced that [\[P0443R14\]](#) (Executors) is sufficiently mature. Library Evolution wants to see more field experience with [\[P0443R14\]](#) (Executors). [\[P0443R14\]](#) (Executors) authors and advocates should take note of this. Focus on demonstrating field experience through implementations and usage, improving introductory material, minimizing the scope, resolving outstanding minor open issues, and developing wording to increase Library Evolution confidence in the maturity of [\[P0443R14\]](#) (Executors).

§ 4.1.6.2. Selected Comments

The executors design that is in [\[P0443R14\]](#) is the result of a long period of hard work and compromise from a large number of authors from various different domains and I feel that it has now settled into a very stable state that well represents the various use cases identified and there has been enough implementation experience that I am confident in the design. I feel that there are still aspects which may need further iteration but a great many alternative approaches have been exhausted in the route to where we are now and I don't believe that the core design should change significantly.

— Strongly Favor

Executors has made a lot of progress over the years and I believe we are on track to ship it in C++23. Assuming that we adopt [\[P2235R0\]](#) (Disentangling Schedulers and Executors), the proposal becomes a lot simpler, and I think once the dust settles from that and other proposed simplifications we will have a solid core proposal.

— Strongly Favor

While I have some concerns about Executors, and there is a chance it may not be ready, I do not believe at this point we should derail this car from the train.

— Strongly Favor

We should certainly (continue to) aim to ship it in C++23.

— Weakly Favor

The executors design discussion has been going on for a long time. It has matured to the point where I think it's important to focus our efforts on shipping it. Focusing on making design decisions that move us closer to shipping in the IS is valuable at this point.

— Strongly Favor

Executors (+schedulers, sender, receiver) are basic building blocks of so many library proposals we MUST aim to ship them in C++23 after missing C++20. It hampers the overall evolution of C++.

— Weakly Favor

I think P0443 is mature enough that we should continue with the goal of landing in C++23. There is a risk that we will work on [\[P0443R14\]](#), but not make C++23, and some other features will miss C++23 because of the time spent on P0443. But I think the benefit of getting executors into C++23 is big enough and the chance of success high enough that it is worth taking that risk.

— Weakly Favor

The recent discussions indicate that, while significant tweaks are justified, the basic design is sound, and prioritizing its (cleanup and) adoption is necessary to meaningfully consider other proposals in flight.

— Weakly Favor

Ideally, I would prefer more usage experience.

— Weakly Favor

I think this still has a chance, but it will be tough. I am particularly concerned about getting the wording into a useful state in time for LWG to get through the paper. It is a bit disheartening to think that it's already "late" into C++23 design when we just released C++20 in February.

— Neutral

Significant aspects of the design seem to be still in flux, and the paper doesn't report implementation/usage experience for the combined facility (even though parts of it have been implemented in different libraries). This seems less than optimal for something as fundamental as executors.

— Neutral

I feel unable to support moving this proposal forward since I still find it next to impossible to understand what's going on, despite extensive discussion and leading a review group.

— Weakly Against

I don't know that the committee fully understands the whole proposal. I don't know that the authors' implementation experience is sufficiently diverse.

— Weakly Against

There are too many things blocked on executors not to try to ship it as soon as possible. The addition of senders and receivers is essential. The use of properties -- which need for which has never been satisfactorily explained -- smells like an immature element. If properties were replaced with `tag_invoke` CPOs would change me to strongly favor.

— Weakly Against

I strongly want executors to make progress towards standardization; however, I am concerned by the number of fundamental changes that have been proposed recently. For example [\[P2235R0\]](#) proposes a design change that completely separates executors / schedulers. There have also been discussions in the last month about whether properties (a large part of [\[P0443R14\]](#)) are the right abstraction. There is also disagreement in SG1 [on whether] algorithms will receive executors or schedulers as arguments. It seems premature to standardize executors/schedulers without knowing what the `std` algorithms will actually use. There is also very little, if any, practical experience with what is specifically proposed in [\[P0443R14\]](#).

— Weakly Against

§ 4.2. Other Polls

§ 4.2.1. Poll 6

Send [\[P2212R1\]](#) (Relax Requirements for `time_point::clock`) to LWG for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
18	13	3	0	0

§ 4.2.1.1. Outcome

Strong consensus in favor.

§ 4.2.1.2. Selected Comments

This is a very small fix which relaxes an unnecessary requirement to support additional valid use cases.

— Strongly Favor

This is a great step towards stateful clocks that are required for simulation and testing faster-than-real-time.

— Weakly Favor

§ 4.2.2. Poll 7

Send [\[P2166R1\]](#) (Prohibit `basic_string` and `basic_string_view` Construction from `nullptr`) to LWG for C++23, classified as an improvement of an existing feature ([\[P0592R4\]](#) bucket 2 item).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
15	18	2	2	1

§ 4.2.2.1. Outcome

Consensus in favor.

§ 4.2.2.2. Selected Comments

This adds a very valuable protection against common errors and makes `string_view` more "self-documenting". The counter-argument that this precludes an eventual permission to allow `string_view` to be constructed from a single null pointer argument seems entirely unhelpful, since such an extension itself would be a poor and unfortunate direction to take (as has been amply discussed, and as was in fact subject of an overwhelmingly clear straw poll).

— Strongly Favor

There is enough benefit to this proposal in preventing bugs that we ought to do it, even though the same bug at runtime may not be caught.

— Weakly Favor

We've discussed this topic many times. We can't fix all the potential footguns, but this covers a decent set of them, so I'm in favor of it.

— Weakly Favor

This is only a partial solution to the problem of creating a string from a null pointer, and the proposed solution has complications.

— Weakly Against

I was doubting about Weakly Against and Strongly Against. But chose the latter. Yes, this proposal is very easy to implement but it's doesn't bring much value to me. It covers very small subset of use-cases. As the paper says it doesn't help if the pointer `p = nullptr` and then we construct `string(p)` and I understand that we cannot do anything useful here at compile-time. What I mean (and the paper explicitly says that) the previous use-case still requires run-time check.

Furthermore, the proposal doesn't look deeply elaborated. It doesn't answer the questions what happens if we construct `string(NULL)` or `string(0)`. With that proposal I am guessing it would cause ambiguity and probably it's better than run-time failure but still doesn't look very obvious to the user. I would like to see other design considerations. For example why don't just mark some templated constructor as `= delete`? I am not saying this is the good solution because it's just taken from the top of my head. But it seems it can cover more use-cases with `nullptr`, `0`, `NULL` and potentially others, which looks better to me. Anyway, other possible solutions should be elaborated and described in the paper. Overall, I think that this proposal needs more work to be done. If the solution for broader number of use-cases wouldn't be found I would not ship this proposal just for the sake of `nullptr` use-case. Currently, I would prefer having some run-time checks (e.g. `assert`) in the debug version of C++ standard library implementations rather than current state of the art of [\[P2166R1\]](#).

— Strongly Against

§ 4.2.3. Poll 8

Send [\[P2161R2\]](#) (Remove Default Candidate Executor) to LWG for the Networking TS Working Draft, classified as a focus work item ([\[P0592R4\]](#) bucket 1 item).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
6	14	2	2	0

§ 4.2.3.1. Outcome

Consensus in favor.

§ 4.2.3.2. *Selected Comments*

The current default executor, `system_executor` makes races to completion implicit in the design and should absolutely not be the default.

— Strongly Favor

This makes it much more difficult to accidentally run afoul of UB when writing networking code. No behavior is lost you just have to opt into the behavior which is always good if the behavior is potentially dangerous.

— Strongly Favor

Executors (in either the Networking TS or in the SG1 incarnation) are meant to provide better control for siting work; making it easy to not exercise such control is not very important and may silently harm performance portability.

— Weakly Favor

Since the executor design and its eventual integration with networking is still in flux, and we do not plan to ship a networking TS v2, forwarding this as a standalone paper at the highest priority seems to be a poor use of limited LWG time. It should be folded into some "integrating executors and networking" paper once the executor design is settled.

— Weakly Against

I think a platform-specific default is useful.

— Weakly Against

§ References

§ Informative References

[P0443R14]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, H. Carter Edwards, Gordon Brown, D. S. Hollman. [A Unified Executors Proposal for C++](https://wg21.link/p0443r14). 15 September 2020. URL: <https://wg21.link/p0443r14>

[P0592R4]

Ville Voutilainen. [To boldly suggest an overall plan for C++23](https://wg21.link/p0592r4). 25 November 2019. URL: <https://wg21.link/p0592r4>

[P1322R2]

Christopher Kohlhoff. [Networking TS enhancement to enable custom I/O executors](https://wg21.link/p1322r2). 21 August 2020. URL: <https://wg21.link/p1322r2>

[P2161R2]

Robert Leahy. [Remove Default Candidate Executor](https://wg21.link/p2161r2). 14 July 2020. URL: <https://wg21.link/p2161r2>

[P2166R1]

Yuriy Chernyshov. [A Proposal to Prohibit std::basic_string and std::basic_string_view construction from nullptr](https://wg21.link/p2166r1). 7 September 2020. URL: <https://wg21.link/p2166r1>

[P2212R1]

Alexey Dmitriev, Howard Hinnant. [Relax Requirements for time_point::clock](https://wg21.link/p2212r1). 14 September 2020. URL: <https://wg21.link/p2212r1>

[P2233R3]

Bryce Adelstein LeLach. [2020 Fall Library Evolution Polls](https://isocpp.org/files/papers/P2233R3.html). 23 November 2020. URL: <https://isocpp.org/files/papers/P2233R3.html>

[P2235R0]

Ville Voutilainen. [Disentangling schedulers and executors](https://wg21.link/p2235r0). 15 October 2020. URL: <https://wg21.link/p2235r0>